

The Second Contact Point in Planar Pushing

Max Gao

*Ira. A. Fulton Schools of Engineering
Arizona State University
Apache Junction, United States of America
mgao18@asu.edu*

Aman Chandak

*Ira. A. Fulton School of Engineering
Arizona State University
Tempe, United States of America
achan160@asu.edu*

I. INTRODUCTION

A simplified yet deeply involved form of non-prehensile manipulation is pushing when restricted to a planar surface. Pushing manipulation, though not as dexterous as true prehensile manipulation, offers many uses in situations where grasping is not realistic. For example, a small coin may be difficult to grasp from a table, while pushing may bring it to a location where it is easier to handle [1]. With the many freedoms of motion still offered by planar motion, pushing manipulation is an explored area of research with promise for larger impact in other fields of object manipulation.

There are many challenges to pushing, despite the two-dimensional nature of the problem. Where manipulation is limited to singular points of contact, the system in question loses some degrees of controllability. This compounds with the many complexities of real-world hybrid dynamics, such as the transitions from static to dynamic friction, leading much research to make simplifying assumptions on physical parameters [1,2]. When applying control theory in the real world, however, errors are all but a guarantee. Thus, robust control is necessary.

One key limitation of planar pushing noted by Zhang et. al. [1] is the set of negative effects of continuous pushing; that is, pushing with one contact point that remains on the object at all times. The alternative presented in Figure 1 is switching pushing, where at a particular time the actuator would release and switch to a different contact point.

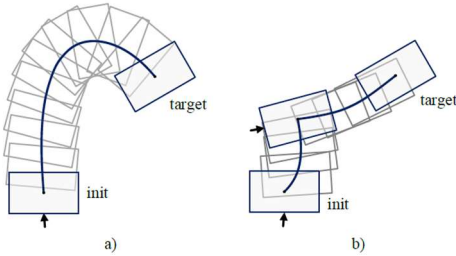


Figure 1. Comparison of continuous pushing (a) and switching pushing (b) [1].

We recognize, however, that many robot end-effectors do not have a singular point of contact. Grippers, like those in Figure 2, can open to create two points of contact, which are naturally at least as effective as one. Therefore, the extent of a second contact point's utility is of interest to us.

Though some works [3] have examined the motion of multiple contact points, the number thereof is typically fixed. However, as seen in Figure 2, both modes of singular and dual

contacts can occur in quick succession. With the loss or gain of one contact, the system's dynamics can change suddenly. This work thus seeks to examine this change of dynamics in the form of an optimization problem, where there are times when the robot must use only one contact.



Figure 2. Example of pushing an allen key on a plane with two (left) or one (right) contact point

II. PROBLEM FORMULATION

To investigate the effectiveness of two points of contact, we have proposed a trajectory-following problem setup, an important execution aspect of motion planning. For a solid object with known physical parameters, a feasible reference trajectory is created, to be followed by controlling a set of pushing contact points.

We suppose that the robot will have two available contacts for a moderate percentage of the full run length T . We formulate this as a mode scheduling problem, where a percentage P of M equal partitions of T will allow use of the second contact, with the remainder only permitting the use of one. $P = 0\%$ implies zero usage of the second contact, while $P = 100\%$ implies that the second contact is always imposing force.

A real-world case of this being necessary involves joint angle limits of the robot's end effector. Imagine, in the left part of Figure 2, the robot gripper is at such a limit and cannot rotate counter-clockwise. To match the angle in the right part of Figure 2, then, it must make nearly a full turn clockwise. For simplification we assume that both contact points static relative to the object over the course of the push, with initial contact positions chosen arbitrarily. In the example of Figure 2, the left contact point stays near the end of the allen key.

Given this constraint, the problem becomes selecting the best schedule for trajectory-following, and comparing results with the two base cases of always having one contact point, and always having two contact points. We also investigate sensitivity to different selections of contact point positions.

III. PHYSICAL MODEL

This work is centered on planar pushing, a simplified form of non-prehensile motion. In planar manipulation, a number of contact points interacts with an object on a flat and level surface, with the goal of sliding the object along a desired path or to a desired end-point.

As our interest lies in a changing number of contact points, it is best to use a kinematic model that handles several forces acting on an object. As such, most of our low-level formulation is derived from the work of Hogan and Rodriguez, who developed a model well-suited for many contact points [2].

A. Kinematics

Examine the system model in Figure 3. A singular object is displaced in an inertial reference frame $\mathcal{F}_a$ by coordinate distances x and y , and oriented by θ . On the body, there are two contact points with angular positions ϕ_1 and ϕ_2 in body frame $\mathcal{F}_b$, acting with normal forces $f_{n,1}$ and $f_{n,2}$, and tangential forces $f_{t,1}$ and $f_{t,2}$.

The quasi-static assumption, which states that frictional forces are in equilibrium, establishes a maximal effective linear force f_{max} and moment m_{max} based on known physical parameters. The maximum moment is generally calculated with the surface integral of r^{dm} over the contact area, though for this work we use a rectangular shape for the object body, which simplifies the calculation.

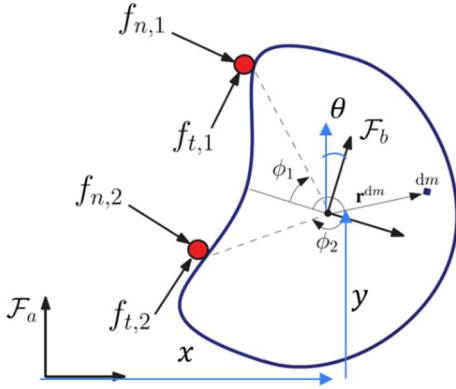


Figure 3: System Dynamic Model [2]

B. Motion Model

One advantage of the quasi-static approximation is that it maps applied force directly to resultant velocity. As is done in [2] and clarified in [4], we use a quadratic approximation to the limit surface, expressed as the equation

$$H(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{L} \mathbf{w}$$

where the scale factor $\mathbf{L}$ is defined as $\text{diag}(2/f_{max}^2, 2/f_{max}^2, 2/m_{max}^2)$ and $\mathbf{w}$ is the applied pushing wrench, or combined linear force and torque.

To establish the force-velocity relationship, we use the principle of maximal dissipation, which states that an object

will move in a way that maximizes power dissipation. This means that the resultant twist $\mathbf{t}$, or combined linear and angular velocity, of the object will be perpendicular to the limit surface. Or,

$$\mathbf{t} = \nabla H(\mathbf{w}) = \mathbf{L} \mathbf{w}$$

This leaves only defining $\mathbf{w}$ in terms of pusher forces, which are split into components of unit normal and tangential vectors $\mathbf{n}_c$, $\mathbf{t}_c$ and force magnitudes $f_{n,c}$, $f_{t,c}$ for each pusher c . A Jacobian

$$\mathbf{J}_c = \begin{bmatrix} 1 & 0 & -y_c \\ 0 & 1 & x_c \end{bmatrix}$$

yields contributions to linear force and moment in the body frame, from pusher positions x_c , y_c and vectors $\mathbf{n}_c$ and $\mathbf{t}_c$, weighted by force magnitudes $f_{n,c}$, $f_{t,c}$. We assume a one-to-one mapping between ϕ_c and (x_c, y_c) is available over the whole body. This gives a coefficient matrix $\mathbf{B}$ in $\mathbf{w} = \mathbf{B} \mathbf{u}$, where:

$$\mathbf{B} = [\mathbf{J}_1^T \mathbf{n}_1, \dots, \mathbf{J}_c^T \mathbf{n}_c, \mathbf{J}_1^T \mathbf{t}_1, \dots, \mathbf{J}_c^T \mathbf{t}_c],$$

$$\mathbf{u} = [f_{n,1}, \dots, f_{n,c}, f_{t,1}, \dots, f_{t,c}]^T$$

Lastly, we apply a rotation matrix $\mathbf{R}(\theta)$ to $\mathbf{t}$, according to the rotation θ of the body about the inertial frame $\mathcal{F}_a$. This gives the full equation of motion:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \mathbf{R}(\theta) \mathbf{t} = \mathbf{R}(\theta) \mathbf{L} \mathbf{B} \mathbf{u}$$

Equation 1: Non-Linear Equation of Motion.

where we define state $\mathbf{x} = [x, y, \theta]^T$.

C. Robot Velocity-Force Mapping

With respect to robot arm control, it is often easier to control the position and velocity of the end-effector than it is to directly control force. Thus, it is important to have a way of resolving end-effector velocity with system control input $\mathbf{u}$. Velocity is related to object velocity and input with the following equation:

$$\mathbf{v}_{p,c} = \mathbf{J}_c \mathbf{L} \mathbf{B} \mathbf{u}$$

where $\mathbf{v}_{p,c}$ is the velocity of the robot pusher, and $\mathbf{J}_c \mathbf{L} \mathbf{B}$ is the coefficient for velocity of the object's contact point.

IV. SYSTEM CONTROL AND OPTIMIZATION

To enable trajectory tracking, we use a Model Predictive Control (MPC) solution, which can effectively look ahead in a linearized form of the dynamic model and optimize controls with user-defined weights.

A. MPC for Trajectory-Following

We use an MPC formulation over a finite time horizon of length N . At each time step t in T , an optimization problem is created with regard to a partial reference trajectory $\{\mathbf{x}_t^*, \dots, \mathbf{x}_{t+N}^*\}$, subject to a series of physical and problem-defined constraints. Over this horizon, the controller seeks to manipulate control $\mathbf{u}$ such that both control magnitude and deviation $\bar{\mathbf{x}} = \mathbf{x} - \mathbf{x}^*$ are reduced. Penalties are weighted by user-defined weights $\mathbf{Q}$ and $\mathbf{R}$, and the quadratic terms are

summed over the length of the horizon. To strengthen the process, a terminal cost is introduced with a stronger weight $\mathbf{Q}_N$.

$$J(\bar{\mathbf{x}}_t, \mathbf{u}_t) = \bar{\mathbf{x}}_{t+N}^\top \mathbf{Q}_N \bar{\mathbf{x}}_{t+N} + \sum_{i=t}^{t+N-1} (\bar{\mathbf{x}}_{i+1}^\top \mathbf{Q} \bar{\mathbf{x}}_{i+1} + \mathbf{u}_i^\top \mathbf{R} \mathbf{u}_i)$$

Equation 2: Trajectory Optimization Formula

We minimize J in Equation 2 subject to the constraints

$$\forall(t, c) \begin{cases} \bar{\mathbf{x}}_{t+1} = \bar{\mathbf{x}}_t + h[\mathbf{A}_t \bar{\mathbf{x}}_t + \mathbf{B}_t \mathbf{u}_t] \\ \sum f_c \leq f_{\max} \\ f_{n,c} \geq 0 \\ \mu f_{n,c} \geq |f_{t,c}| \end{cases}$$

Equation 3: Dynamics Constraints applied to each pusher, at every time step.

where h is the discretization time step, and $\mathbf{A}_t$ and $\mathbf{B}_t$ are state dynamic matrices for the linearized system $\dot{\bar{\mathbf{x}}}_t = \mathbf{A}_t \bar{\mathbf{x}}_t + \mathbf{B}_t \mathbf{u}_t$, which are computed as:

$$\mathbf{A}_t = \left. \frac{\partial(\mathbf{R}(\theta)\mathbf{L}\mathbf{B}\mathbf{u})}{\partial \mathbf{x}} \right|_{\mathbf{x}_t^*, \mathbf{u}_t^*} = \mathbf{u}_t^* = \mathbf{0},$$

$$\mathbf{B}_t = \left. \frac{\partial(\mathbf{R}(\theta)\mathbf{L}\mathbf{B}\mathbf{u})}{\partial \mathbf{u}} \right|_{\mathbf{x}_t^*, \mathbf{u}_t^*} = \mathbf{R}(\theta_t^*)\mathbf{L}\mathbf{B}$$

Equation 4: Linearization of Equation 1

Note that in Equation 4, θ_t^* is from $\mathbf{x}_t^* = [x_t^*, y_t^*, \theta_t^*]^\top$. The latter three physics constraints in Equation 3 pertain to satisfying the quasi-static assumption, preventing negative normal force, and preventing contact point sliding. μ in the final constraint is the coefficient of friction between the object surface and the pusher.

Lastly, we constrain f_c according to the contact mode plan, which determines at which t that $f_c = 0$. The contact mode plan is given at the start of each experiment, and is discussed in the next part.

As the cost function is quadratic, and as all of these constraints are either affine or convex, the entire optimization problem over the horizon can be solved in real-time. This can be done with a variety of openly available solver tools, such as OSQP.

B. Optimal Mode Scheduling

The overall objective is to determine a best-performing mode plan, using the MPC method and constrained by design parameters. This plan takes the form

$$\mathcal{P} = \{(\phi_{1,1}, \dots, \phi_{C,1}, p_{1,1}, \dots, p_{C,1}), \dots, (\phi_{1,T}, \dots, \phi_{C,T}, p_{1,T}, \dots, p_{C,T})\}$$

where $p_{c,t} = 0$ specifies $f_c = 0$ at step t , and $p_{c,t} = 1$ does not.

The most straightforward approach for a two-pusher setup, as mentioned in section II, is to split the total time length T in to M bins of equal length, clarify that $p_{1,t} = 1$ for all t , and set a number of bins such that for all t falling inside of them, $p_{2,t} = 0$. All ϕ remain constant.

For small M , the schedule space of size 2^M can be searched exhaustively as MPC is fast. Therefore, we explore the additional effects of varying ϕ , as this generally has strong impacts on MPC performance.

V. EXPERIMENTAL SETUP

All of our initial MPC testing was done with numerical simulation using the nonlinear dynamics in Equation 1. We use a rectangular block as the body, with side lengths of 0.1 m by 0.05 m, mass 0.8 kg, ground μ as 0.35, and pusher μ as 0.3. For MPC, we set $\mathbf{Q} = \text{diag}(30, 30, 1)$ to prioritize tracking position, $\mathbf{Q}_N = 200\mathbf{Q}$ to strongly emphasize following the reference, and $\mathbf{R} = 0.2\text{diag}(1, 1, 1)$ to de-prioritize force regulation. All simulations had a length of $T = 300$ time steps with $h = 0.05$ s for discretization and a horizon of $N = 30$ time steps. For faster solving, only every third time step of this horizon was included in MPC optimization, with minimal tradeoffs in accuracy.

State initial conditions were $x = [0.4, 0.3, 0]^\top$, aligning the block exactly with the start of its reference trajectory. The reference trajectory is defined as a circle of radius 0.1 m, with the added requirement that the block should make one full turn over its course. In our experiments, we placed two pushers at $\phi = \frac{15\pi}{12}, \frac{21}{12}$, which are respectively the leftmost and rightmost red dots in Figure 4. In the first set of experiments, $\phi_1, \phi_2 = \frac{21}{12}, \frac{15\pi}{12}$, and vice versa for the second.

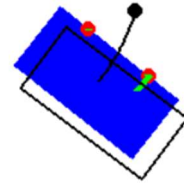


Figure 4: Basic environmental testbed, in Pygame. Contact points are marked as red circles, forces are green lines, and the past trajectory is drawn in black (starting from the dot). The black rectangle is the reference position at the current time step.

Two sets of 256 simulations were run, each exploring all possible mode schedules for $M = 8$ equally-sized bins. In each simulation, the full trajectory and total cost $J = \sum_{i=0}^{N-1} (\bar{\mathbf{x}}_{i+1}^\top \mathbf{Q} \bar{\mathbf{x}}_{i+1} + \mathbf{u}_i^\top \mathbf{R} \mathbf{u}_i)$ were recorded. Results were grouped into plans of equal usage of pusher 2, as percentages $P = 0, 12.5, \dots, 100$.

To verify the function of the controller outside of the simplistic physical model, a higher-fidelity simulation environment was built in MuJoCo, a digital twin software made for applications in robotics. To attempt to match our physical

hardware, we have sourced a model of a UR5 robot arm and have demonstrated basic pushing of a block.

As gripper model files were difficult to source in time, however, we have opted for a middle-ground simplification of two spheres acting as point pushers, with many of the intended physical interactions still being captured. The environment is shown in Figure 5.

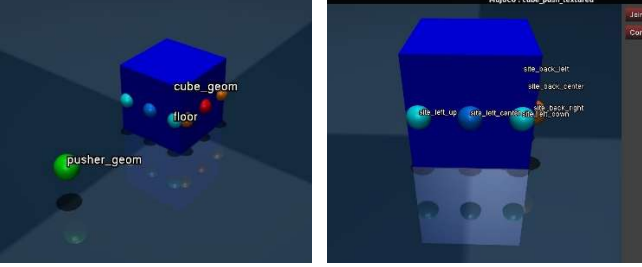


Figure 5: Digital Twin Environmental Testbed, in MuJoCo

Lastly, we have secured use of a real-world experimental environment, with a Universal Robots (UR) 5e collaborative robot arranged to conduct tests with planar pushing. We have completed work on end-effector inverse kinematics and object detection, with an object-following script recorded and demonstrated, however work has fallen closely short of porting the MPC script to the real world. Figure 6 shows an uncontrolled interaction with a wooden block, as seen from a camera mounted on the end-effector.

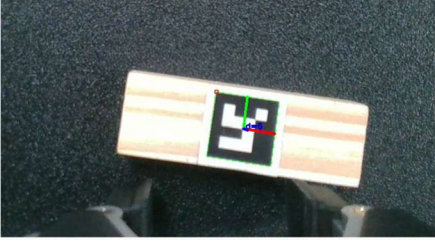


Figure 6: Top-down view of wooden block, tracked with the aid of an ArUco marker.

VI. EXPERIMENTAL RESULTS

A. Simulation Experiment Set 1: $\phi_1, \phi_2 = \frac{21\pi}{12}, \frac{15\pi}{12}$

This first set of ϕ was picked such that the general shape of the trajectory could be captured even with only one pusher. As shown in Figure 7, however, the stabilizing effect of the second pusher is not insignificant, if timed well. In general, as seen in Figure 8, having the second pusher be active for longer will result in lower overall costs.

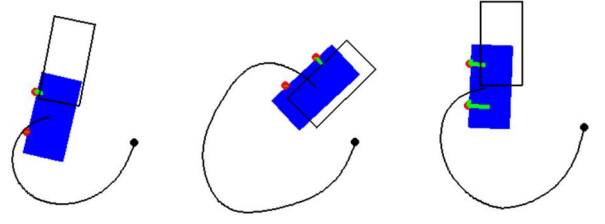


Figure 7: Three sample simulations representing the only case of $P = 0\%$ (left), the best case of $P = 25\%$ (middle), and the worst case of $P = 50\%$ (right). Of these three, as seen in Figure 8, the middle simulation achieved the best score. When comparing with later figures, note that Pygame flips all visuals vertically.

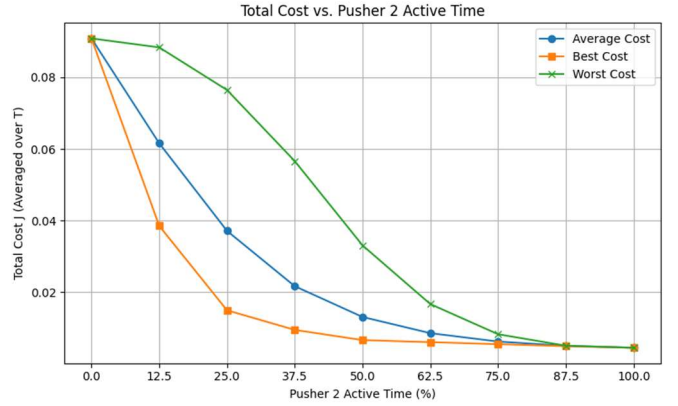


Figure 8: Comparison between different values of P in terms of best, average, and worst total cost. As expected, higher values of P will always lead to smaller J , but there are diminishing returns at around $P = 25\%$.

The rationale behind the sharp dropoff in best total cost is explained in Figure 9, which displays the best pusher plan per value of P . We see here and in Figures 10 and 11 that if error growth can be halted with correction from the second pusher, the overall cost can be reduced substantially.

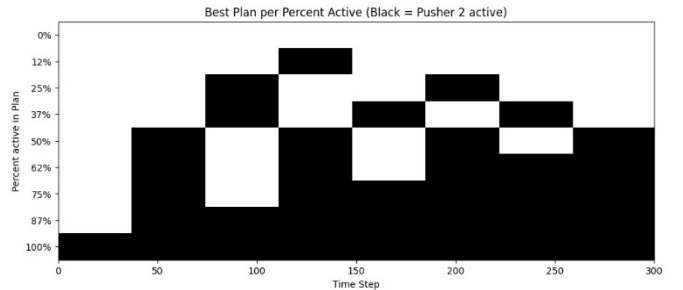


Figure 9: Block visualization of best pusher plan, for each P . In each row, white signifies $p_2 = 0$, and black signified $p_2 = 1$. In general, when given m bins of $M = 8$, the best approach is to space them out evenly, as to minimize the longest continual period of having only one pusher.

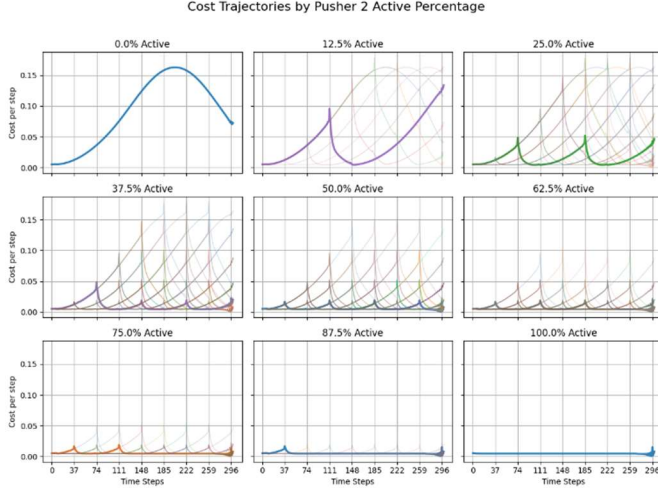


Figure 10: Best cost trajectory for each value of P . The cost-accumulation effect of having only one pusher is evident for lower P , as underactuation fails to completely track the reference trajectory. Small spikes are seen at the peaks of each cost plot, signifying sudden increases in control cost, though the cost savings over the horizon is much greater.

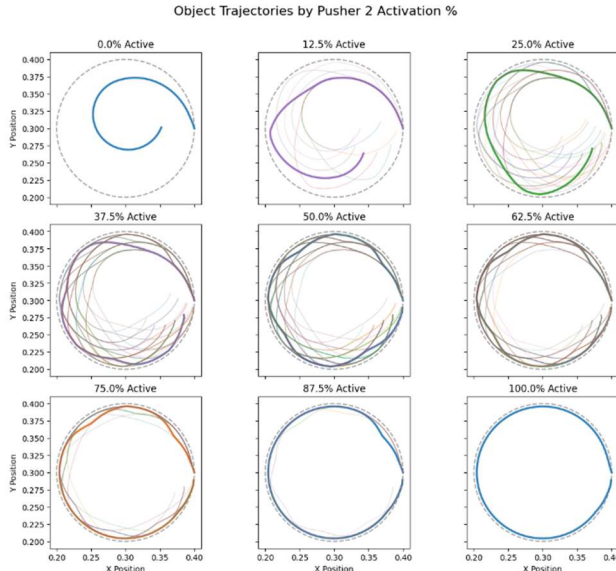


Figure 11: Best vs. Reference trajectory for each value of P . Mirroring the results in Figure 10, with only one pusher, the trajectory has a natural tendency to curve inward. Interrupting this with intermittent corrections from the second pusher is enough to recover a circular path.

Lastly in Figure 12 we explore the relationship between force usage in pusher 1 and in pusher 2, with the prior expectation that where pusher 2 is unavailable, pusher 1 may need to push more to compensate. We see this is not the case, and that under an MPC controller, a single pusher will severely limit its output as to not worsen the buildup of error seen previously. Instead, best results are achieved if both pushers are at their allowable strongest.

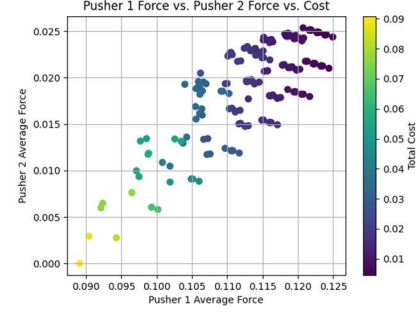


Figure 12: Pusher force comparison, with total cost. It is of interest that the force outputs of the two pushers are positively correlated, with a natural explanation being that the two should be balanced as to not rotate the block too much.

B. Simulation Experiment Set 2: $\phi_1, \phi_2 = \frac{15\pi}{12}, \frac{21\pi}{12}$

In this set of experiments, we simply reverse which of the two pushers is turned on and off throughout the course of the simulation. We notice that though many patterns of optimal scheduling remain the same, some interesting interactions emerge from the controller's inability to track with only one pusher. Comparing Figures 8 and 13, one obvious effect is a massive increase in cost.

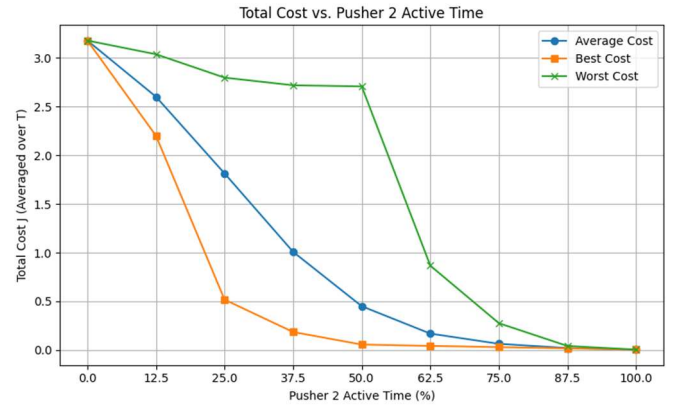


Figure 13: Comparison between values of P for Experiment Set 2. Seeing the Experiment Set 1 had a maximum cost of about 0.09, we see it takes until $P = 50\%$ for the controller to achieve the same. Even then, as seen by the worst cost and in Figure 14, tracking the reference is not outright guaranteed.

As the error accumulation effect in this experiment set is far more severe than that discussed in the prior set, full tracking of the reference is only achieved with plans of $P \geq 37.5\%$ as seen in Figure 14, and even then with interesting behavior as seen in Figure 15. However, we see that with higher values of P , the plans approximate the counterparts in Experiment Set 1, as the dynamics of both pushers being active in that set is the same as in this set.

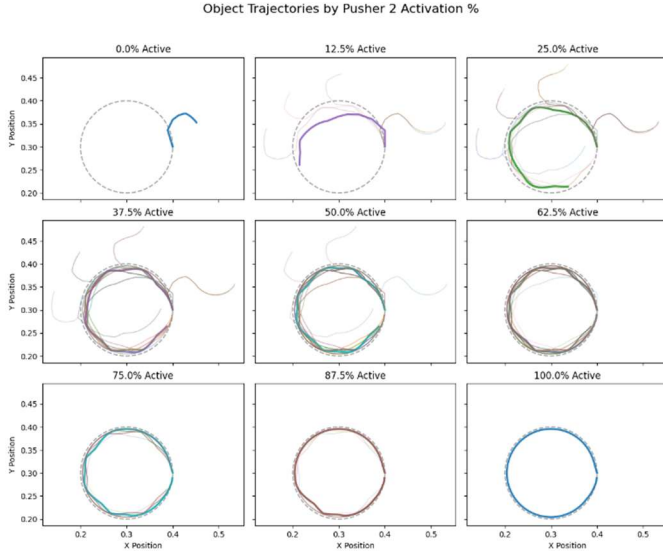


Figure 14: Best vs. Reference trajectory for each value of P for Experiment Set 2. Wisps leading away from the circle are trajectories that badly failed to track the reference. By $P = 62.5\%$, however, these wisps are gone.

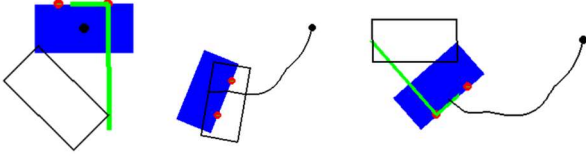


Figure 15: Snapshots of the best controller in $P = 37.5\%$.

Since pusher 1 cannot by itself track the reference, the controller waits for pusher 2 to become available, then *moves the block ahead of schedule*, so that it may hold position as the reference catches up.

C. Simulation in MuJoCo

Encouraged by the success of the MPC solution in the previous experiment sets, we have proceeded to implement a controller variant in MuJoCo, using a singular point pusher to have a cube follow a sample rectilinear reference trajectory. As can be seen in Figures 16 and 17, the controller was able to adapt to a more realistic dynamic system and track the reference.

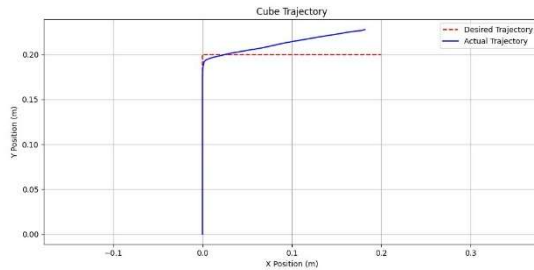


Figure 16: MPC results in the MuJoCo simulation, with simulated vs reference trajectory.

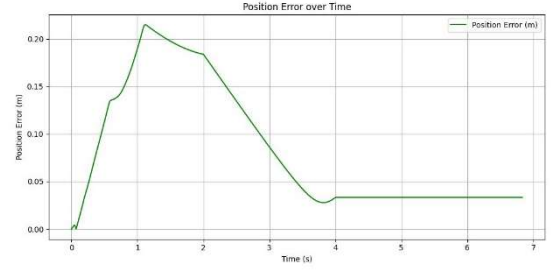


Figure 17: MPC results in the MuJoCo simulation, with position error over time.

VII. REFLECTIONS AND FURTHER WORK

We have presented a novel problem formulation using a finite-horizon MPC controller for variable pusher systems. When contact points are placed well, our MPC is effective at tracking challenging reference trajectories, even with one pusher being partially unavailable. As this controller can be tuned to run in real time, it can be freely implemented in physical systems such as the UR5 testbed shown earlier.

We see that not only is our MPC robust when contact point locations are chosen well, it can also to a degree handle poorly-arranged contact points, where the first pusher alone will only bring the block out of the circle. The optimization over a horizon exhibits greater intelligence by waiting for the second pusher to come online, then quickly maneuvering the block using the greater actuation afforded to it.

Lastly, through our simulation work in MuJoCo, we have demonstrated that our MPC controller is able to function with object kinematics beyond the simplistic physical model constructed in section III. With this, we have higher confidence in implementing a successful controller in the real world.

Though we have already seen interesting dynamics with the varying of contact point location between experiments, we most intend to next explore the changing of contact points inside experiments. Through these expansions, we seek to make a push controller that makes the absolute most of the actions affordable to it.

REFERENCES

- [1] B. Zhang, C. Huang, H. Zhang, X. Bai. "Switching Pushing Skill Combined MPC and Deep Reinforcement Learning for Planar Non-prehensile Manipulation." *arXiv preprint arXiv:2303.17379* (2023).
- [2] F. R. Hogan, A. Rodriguez. "Reactive planar non-prehensile manipulation with hybrid model predictive control." *The International Journal of Robotics Research*. 39(7):755-773 (2020), doi:10.1177/0278364920913938.
- [3] K. M. Lynch. "Locally controllable manipulation by stable pushing." *IEEE Transactions on Robotics and Automation*, 15(2):318-327 (1999), doi: 10.1109/70.760353.
- [4] Y. Lee, K. Kim. "Accurate Robotic Pushing Manipulation Through Online Model Estimation Under Uncertain Object Properties." *IEEE Robotics and Automation Letters*, 9(10): 8730-8737 (2024), doi: 10.1109/LRA.2024.3449695.

All code used in making the MPC controller and Pygame simulator can be found here: <https://github.com/MaxiGao/-EEE587>